







Public repository · Forked from [mate-academy/py-flip-coin](#)

This branch is up to date with `mate-academy/py-flip-coin:master`.

Sync fork

f7a1822 · 6 months ago

 .github/workflows	add auto-approve and tests	3 years ago
 app	Flip coin	3 years ago
 tests	upd tests	3 years ago
 .flake8	Update .flake8	3 years ago
 .gitignore	Initial commit	3 years ago
 README.md	Update README.md	2 years ago
 requirements.txt	Update requirements.txt	6 months ago

- Read [the guideline](#) before start

Write `flip_coin` function, that conducts at least 10000 cases of flipping a coin 10 times. It should return dict, where keys are numbers of possible heads dropped (0 to 10), and values are percentage of how many that number of heads dropped out of all cases.



If you have done all correctly, you should note that the biggest percentage of a number of heads dropped is about the middle, 4-6 times and tends to 0 when it is about the edges 0-1 and 9-10. It calls normal distribution or Gaussian distribution.

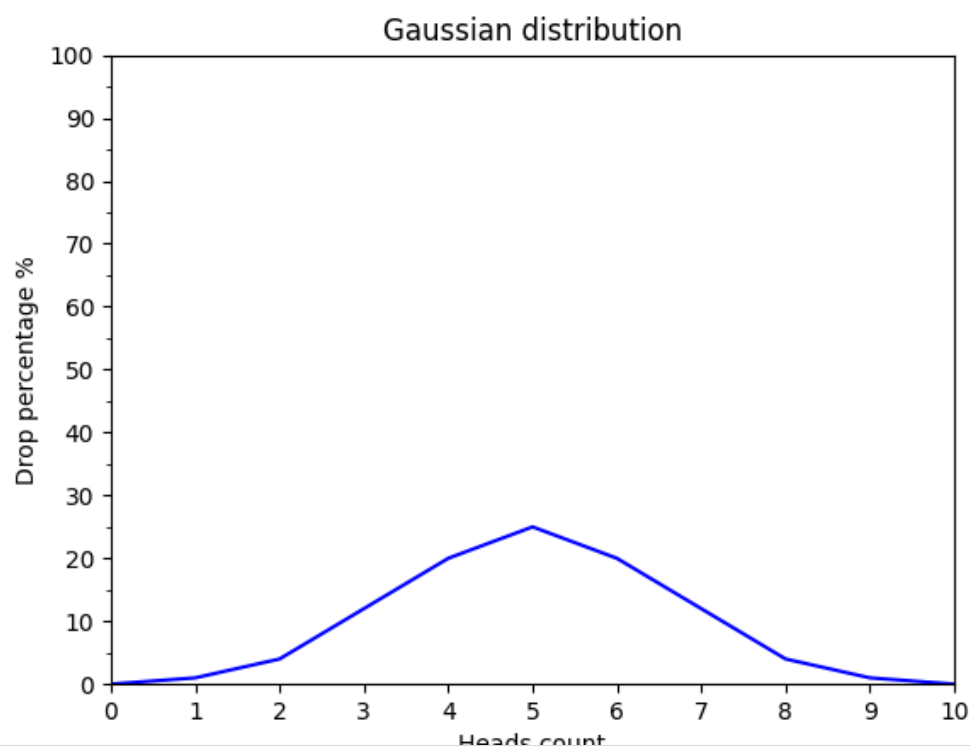
Extra

Write `draw_gaussian_distribution_graph` function, that draws a graph that shows the distribution.

`matplotlib` is relevant library for this purpose.

- [matplotlib, Plotting](#)
- [matplotlib, Labels](#)

You should get graph like this:



Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Languages

● Python 100.0%

Suggested workflows

Based on your tech stack



Publish Python Package
Publish a Python Package to PyPI on release.

Configure



SLSA Generic generator
Generate SLSA3 provenance for your existing release workflows

Configure



Python Package using Anaconda
Create and test a Python package on multiple Python versions using Anaconda for package management.

Configure

[More workflows](#)

[Dismiss suggestions](#)



Matplotlib Plotting

[< Previous](#)[Next >](#)

Plotting x and y points

The `plot()` function is used to draw points (markers) in a diagram.

By default, the `plot()` function draws a line from point to point.

The function takes parameters for specifying points in the diagram.

Parameter 1 is an array containing the points on the **x-axis**.

Parameter 2 is an array containing the points on the **y-axis**.

If we need to plot a line from (1, 3) to (8, 10), we have to pass two arrays [1, 8] and [3, 10] to the plot function.

Example

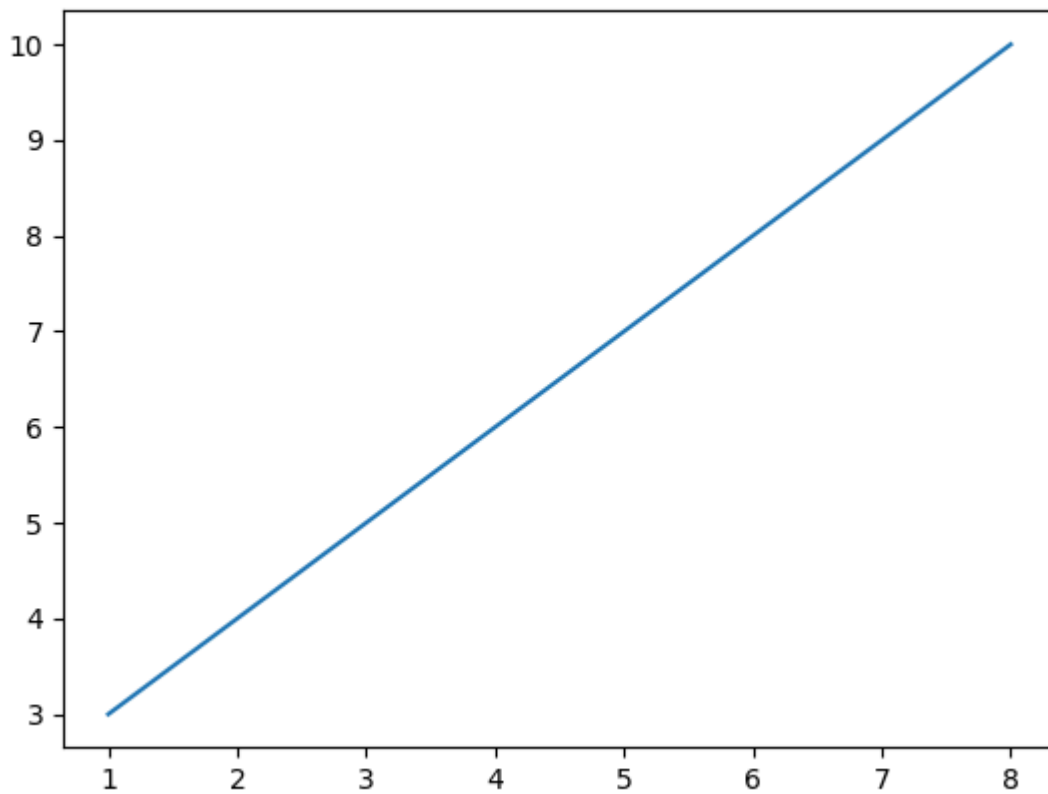
[Get your own Python Server](#)

Draw a line in a diagram from position (1, 3) to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints)
plt.show()
```



Try it Yourself »

The **x-axis** is the horizontal axis.

The **y-axis** is the vertical axis.

ADVERTISEMENT

[REMOVE ADS](#)

Plotting Without Line

To plot only the markers, you can use *shortcut string notation* parameter 'o', which means 'rings'.

Example

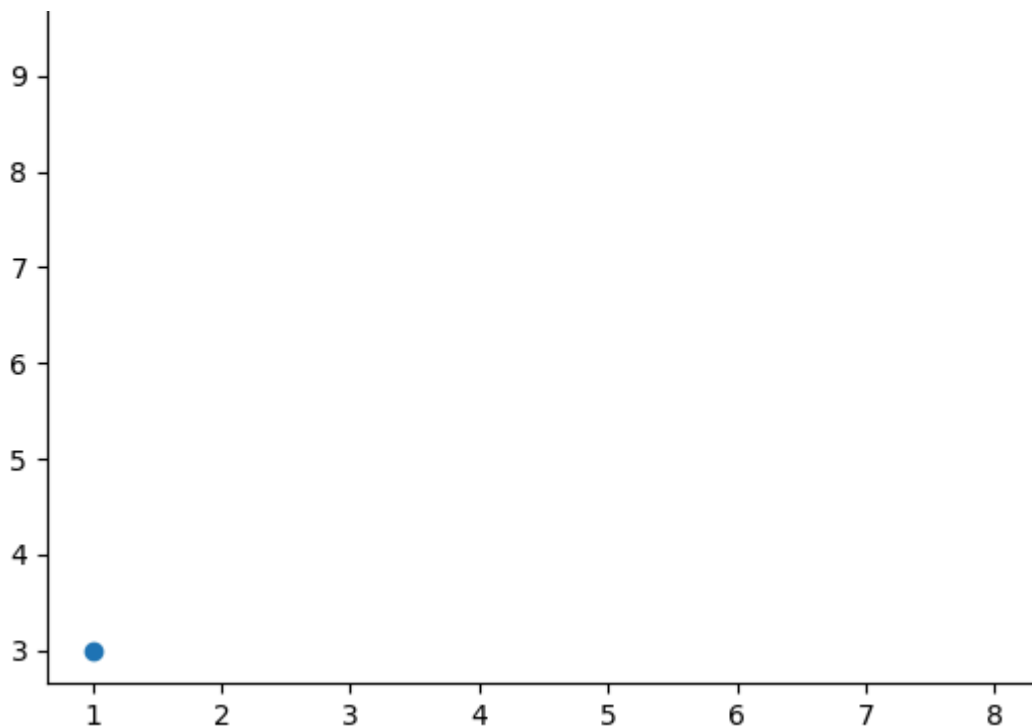
Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints, 'o')
plt.show()
```

Result:



[Try it Yourself »](#)

You will learn more about markers in the next chapter.

Multiple Points

You can plot as many points as you like, just make sure you have the same number of points in both axis.

Example

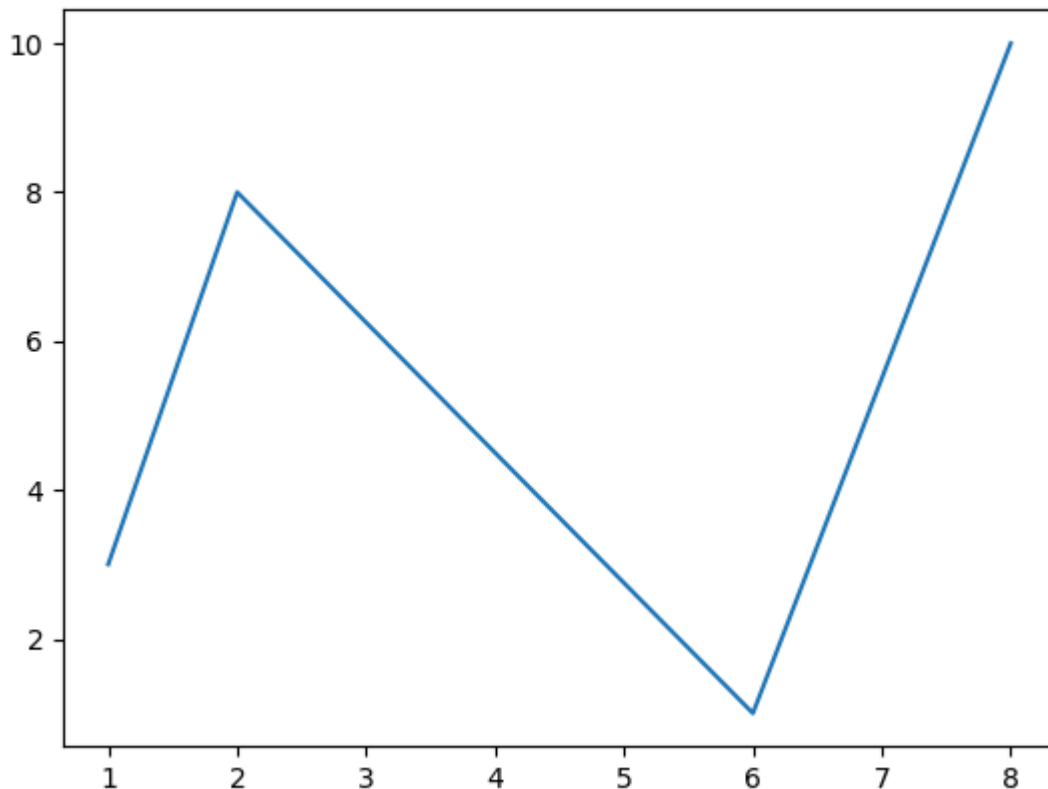
Draw a line in a diagram from position (1, 3) to (2, 8) then to (6, 1) and finally to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np
```



```
plt.plot(xpoints, ypoints)  
plt.show()
```

Result:



[Try it Yourself »](#)

Default X-Points

If we do not specify the points on the x-axis, they will get the default values 0, 1, 2, 3 etc., depending on the length of the y-points.

So, if we take the same example as above, and leave out the x-points, the diagram will look like this:

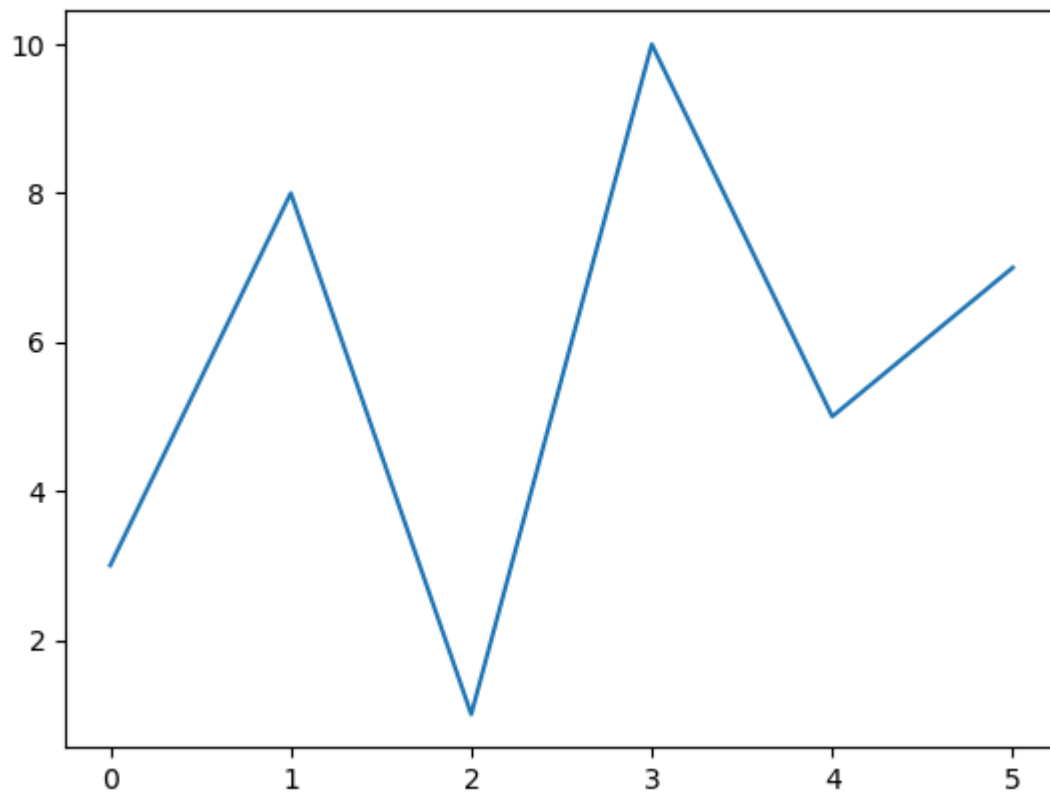


```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10, 5, 7])

plt.plot(ypoints)
plt.show()
```

Result:



Try it Yourself »

The **x-points** in the example above are [0, 1, 2, 3, 4, 5].



Matplotlib Labels and Title

[< Previous](#)[Next >](#)

Create Labels for a Plot

With Pyplot, you can use the `xlabel()` and `ylabel()` functions to set a label for the x- and y-axis.

Example

[Get your own Python Server](#)

Add labels to the x- and y-axis:

```
import numpy as np
import matplotlib.pyplot as plt

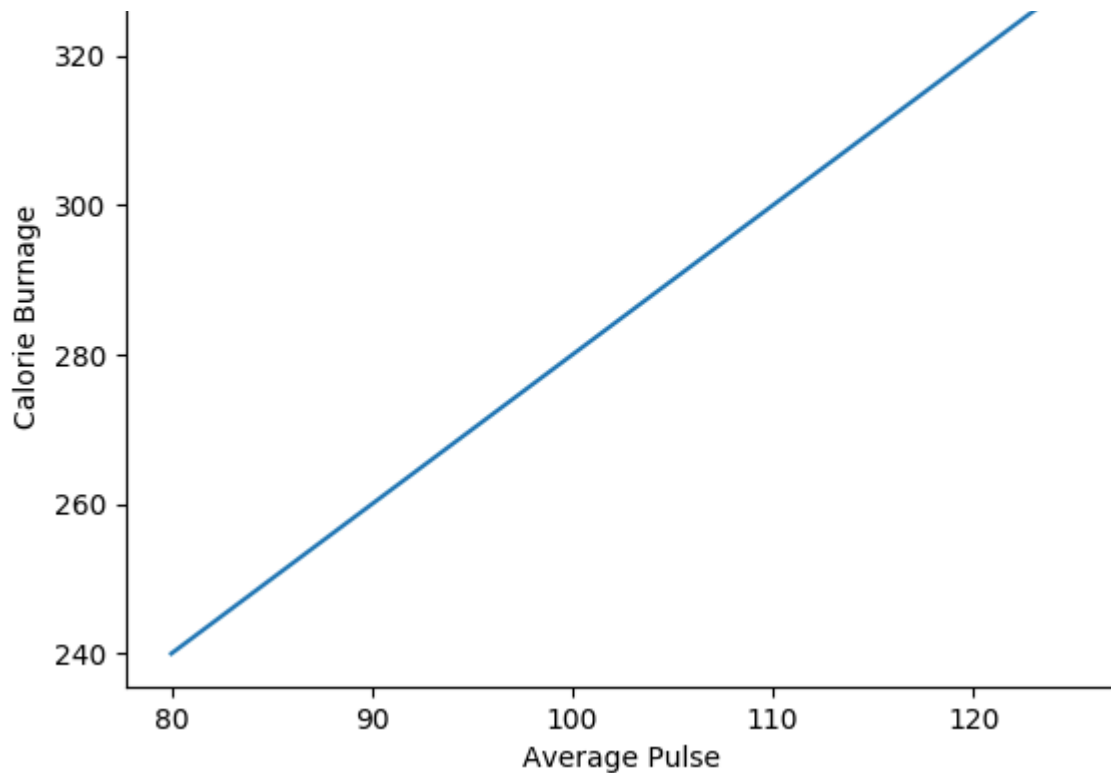
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

Result:



[Try it Yourself »](#)

Create a Title for a Plot

With Pyplot, you can use the `title()` function to set a title for the plot.

Example

Add a plot title and labels for the x- and y-axis:

```
import numpy as np
import matplotlib.pyplot as plt

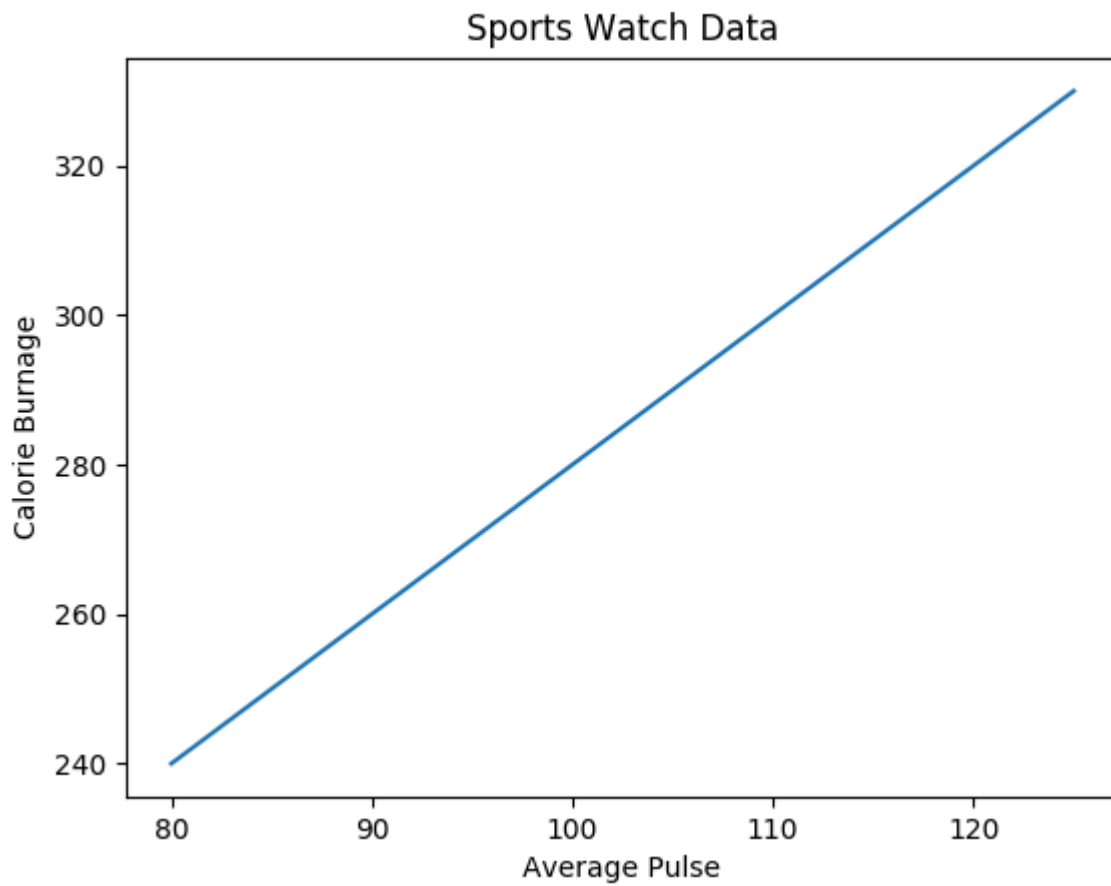
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.title("Sports Watch Data")
```



Result:



[Try it Yourself »](#)

ADVERTISEMENT



Set Font Properties for Title and Labels

You can use the `fontdict` parameter in `xlabel()`, `ylabel()`, and `title()` to set font properties for the title and labels.

Example

Set font properties for the title and labels:

```
import numpy as np
import matplotlib.pyplot as plt

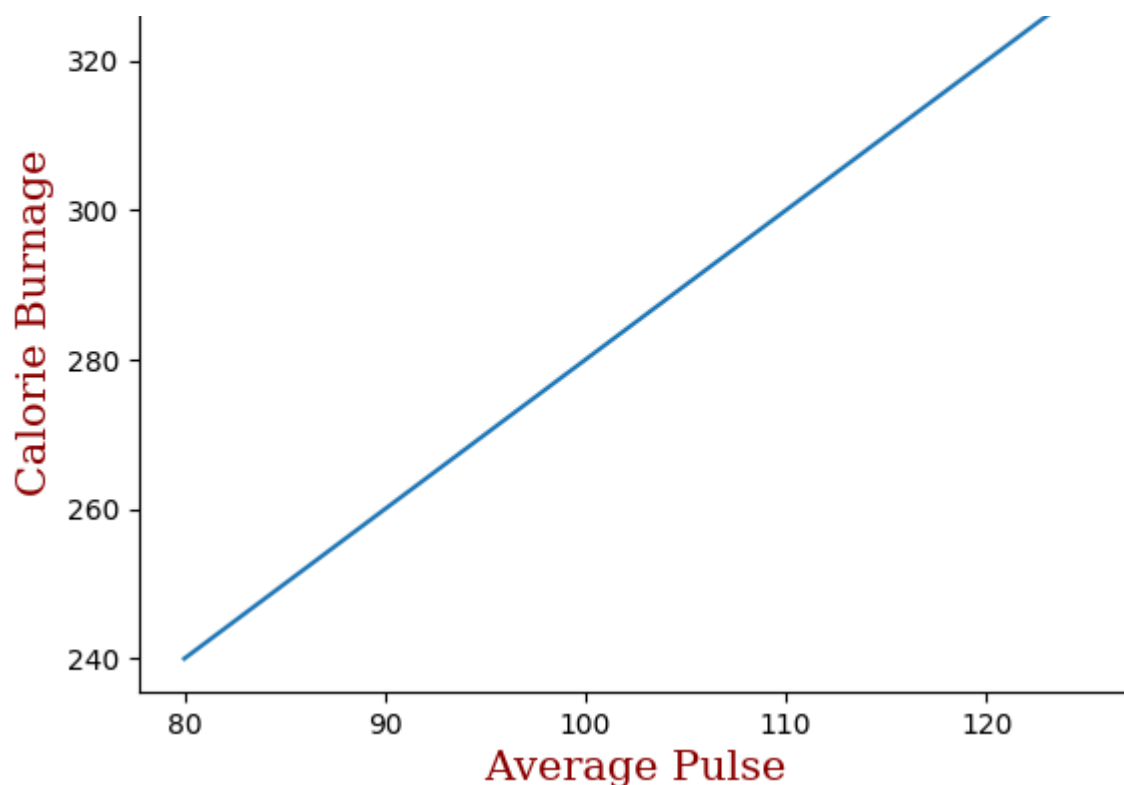
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

font1 = {'family':'serif','color':'blue','size':20}
font2 = {'family':'serif','color':'darkred','size':15}

plt.title("Sports Watch Data", fontdict = font1)
plt.xlabel("Average Pulse", fontdict = font2)
plt.ylabel("Calorie Burnage", fontdict = font2)

plt.plot(x, y)
plt.show()
```

Result:



[Try it Yourself »](#)

Position the Title

You can use the `loc` parameter in `title()` to position the title.

Legal values are: 'left', 'right', and 'center'. Default value is 'center'.

Example

Position the title to the left:

```
import numpy as np
import matplotlib.pyplot as plt

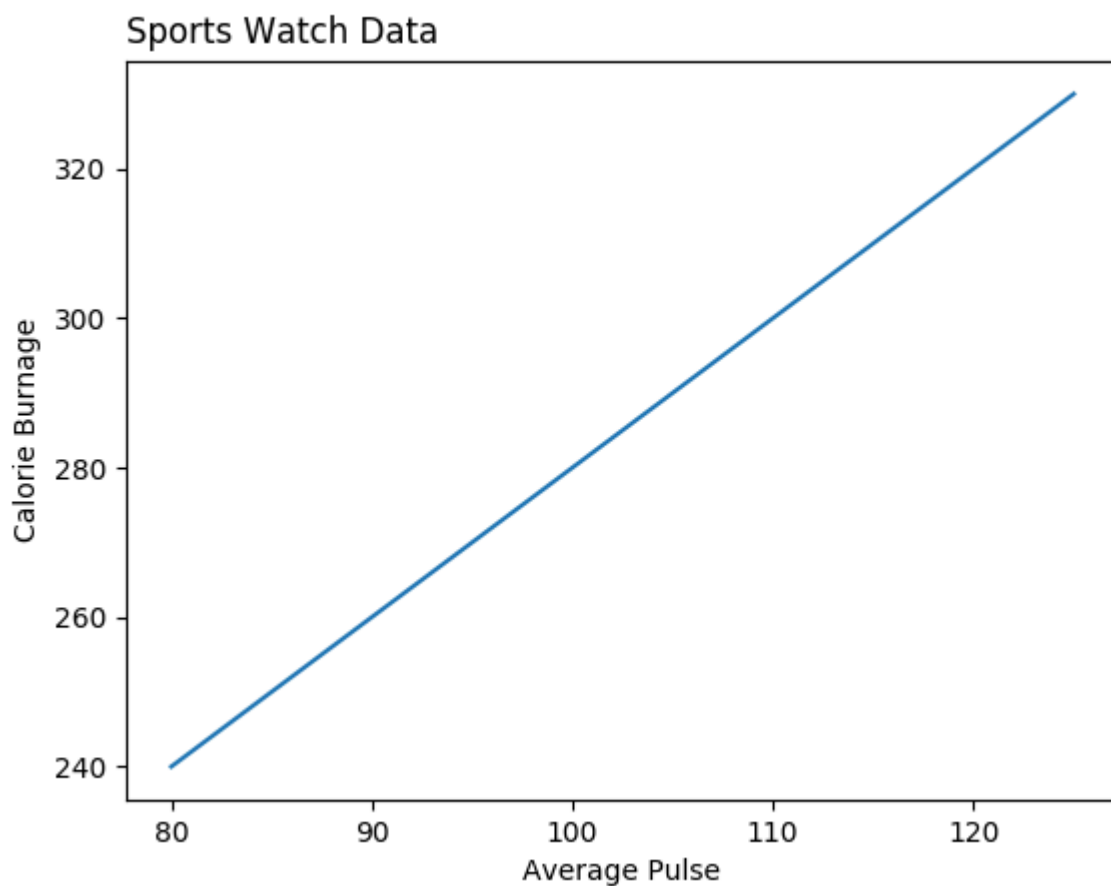
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data", loc = 'left')
```

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#)

```
plt.show()
```

Result:



[Try it Yourself »](#)

[◀ Previous](#)[Sign in to track progress](#)[Next >](#)

ADVERTISEMENT